# ALPHAEDIT: NULL-SPACE CONSTRAINED KNOWLEDGE EDITING FOR LANGUAGE MODELS

**Junfeng Fang**[1,2]*, **Houcheng Jiang**[1]*, **Kun Wang**[1], **Yunshan Ma**[2],
**Jie Shi**[2], **Xiang Wang**[1]†, **Xiangnan He**[1]†, **Tat-Seng Chua**[2]
[1]University of Science and Technology of China, [2]National University of Singapore
fangjf1997@gmail.com, jianghc@mail.ustc.edu.cn

arXiv:2410.02355v4 [cs.CL] 22 Apr 2025

## ABSTRACT

Large language models (LLMs) often exhibit hallucinations, producing incorrect or outdated knowledge. Hence, model editing methods have emerged to enable targeted knowledge updates. To achieve this, a prevailing paradigm is the locating-then-editing approach, which first locates influential parameters and then edits them by introducing a perturbation. While effective, current studies have demonstrated that this perturbation inevitably disrupt the originally preserved knowledge within LLMs, especially in sequential editing scenarios. To address this, we introduce AlphaEdit, a novel solution that projects perturbation onto the null space of the preserved knowledge before applying it to the parameters. We theoretically prove that this projection ensures the output of post-edited LLMs remains unchanged when queried about the preserved knowledge, thereby mitigating the issue of disruption. Extensive experiments on various LLMs, including LLaMA3, GPT2-XL, and GPT-J, show that AlphaEdit boosts the performance of most locating-then-editing methods by an average of 36.7% with **a single line of additional code for projection solely**. Our code is available at: https://github.com/jianghoucheng/AlphaEdit.

## 1 INTRODUCTION

Large language models (LLMs) have demonstrated the capability to store extensive knowledge during pre-training and recall it during inference (Brown et al., 2020; Petroni et al., 2019; Roberts et al., 2020; Liu et al., 2024). Despite this, they frequently exhibit hallucinations, producing incorrect or outdated information (Cao et al., 2021; Mitchell et al., 2022a). While fine-tuning with updated knowledge offers a straightforward solution, it is often prohibitively time-consuming (Mitchell et al., 2022b). In sight of this, model editing methods have emerged, enabling updating the target knowledge while preserving other knowledge (Yao et al., 2023; Gupta et al., 2024). Broadly, model editing approaches fall into two categories: (1) parameter-modifying methods, which directly adjust a small subset of parameters (Meng et al., 2023; Jiang et al., 2025), and (2) parameter-preserving methods that integrate additional modules without altering the original parameters (Huang et al., 2023; Yu et al., 2024; Hartvigsen et al., 2023; Zheng et al., 2023).

In this paper, we aim to explore the parameter-modifying methods for model editing. Concretely, current parameter-modifying methods typically follow the locate-then-edit paradigm (Meng et al., 2022). The basic idea is to first locate influential parameters $\boldsymbol{W}$ through causal tracing, and then edit them by introducing a perturbation $\boldsymbol{\Delta}$ (Li et al., 2024b). The common objective for solving $\boldsymbol{\Delta}$ is to minimize the output error on the to-be-updated knowledge, denoted as $e_1$. Additionally, the output error on the to-be-preserved knowledge, $e_0$, is typically incorporated into the objective function, acting as a constraint to ensure the model's accuracy on the preserved knowledge.

Despite their success, the current paradigm faces a critical limitation: it struggles to maintain a balance between knowledge-update error $e_1$ and knowledge-preservation error $e_0$. Specifically, to prioritize the success of update, prior studies focus more on minimizing $e_1$ by assigning a larger weight, while taking insufficient control over $e_0$. This could make the LLMs after editing (*i.e.,* the

*Equal contribution. †Corresponding author: {xiangwang1223, xiangnanhe}@gmail.com.

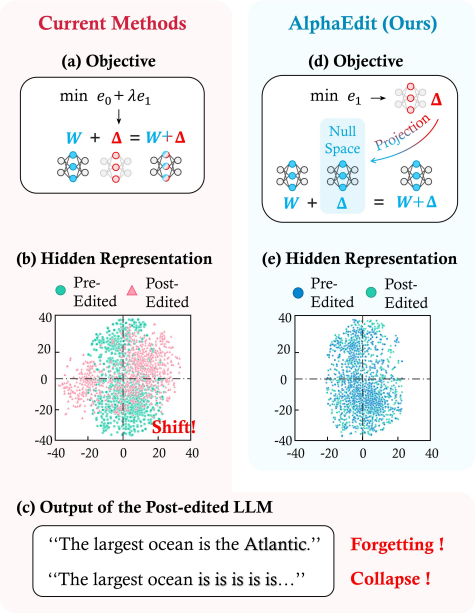


Figure 1: Comparison between the current methods and AlphaEdit. (a) and (d) exhibit the objectives, where $\lambda$ is the coefficient to keep balance between $e_0$ and $e_1$ in the objective; (b) and (e) show the distributions of hidden representations after dimensionality reduction within the pre-edited and post-edited LLaMA3, respectively; (c) depicts the output of the post-edited LLaMA3. Best viewed in color.

Figure 2: Performance of various model editing methods on LLaMA3 (8B). Results with asterisks in the superscript are from the ZsRE dataset. SST, RTE, and CoLA demonstrate the general capabilities of the post-edited LLMs. The experiments are conducted with 2000 edited samples for sequential editing. Detailed settings and results are provided in Section 4.2 and Table 1, respectively. Best viewed in color.

post-edited LLMs) overfit to the updated knowledge. This overfitting would introduce a distribution shift of the hidden representations within LLMs. Figure 1 (b) showcases this shift, where the hidden representations in the post-edited LLaMA-3 (8B) (Meta, 2024) diverge from their distribution in the original LLM (*i.e.,* the pre-edited LLM). Worse still, in sequential editing scenarios (Gupta & Anumanchipalli, 2024) where the LLM undergoes multiple sequential edits, the accumulation of overfitting gradually erodes the model's ability to preserve knowledge and generate coherent sentences, eventually leading to model forgetting and model collapse, as depicted in Figure 1 (c).

To address these flaws, we instead remove $e_0$ from the current objective, allowing the model to focus solely on minimizing $e_1$ without trade-offs. To avoid overfitting to the to-be-updated knowledge, we project the solution of this new objective onto the null space (Wang et al., 2021) of the preserved knowledge before applying it to the model parameters, as shown in Figure 1 (d). By leveraging the mathematical properties of matrix projection and null space, our new objective ensures that the distribution of hidden representations within LLMs remains invariant post-edited, as shown in Figure 1 (e). This invariance enables the post-edited LLMs to reduce $e_1$ while keeping $e_0$ close to zero, thereby alleviating the issues of model forgetting and model collapse. A detailed theoretical proof is provided in Section 3. In a nutshell, we term the method as **AlphaEdit**, a simple yet effective editing approach with a null-space constraint for LLMs.

To validate the effectiveness of our method, we conducted extensive experiments on multiple representative LLMs, such as GPT-2 XL (Radford et al., 2019) and LLaMA-3 (8B). The results show that, compared to the best-performing baseline methods, **AlphaEdit can achieve over a 36.7% performance improvement on average by adding just one line of code** to the conventional model editing method, MEMIT (Meng et al., 2023), as illustrated in Figure 2. Furthermore, we empirically verified that this simple optimization can be easily integrated into most existing model editing methods (Ma et al., 2024; Gu et al., 2024), functioning as a plug-and-play enhancement that significantly boosts their performance. This highlights AlphaEdit's crucial role in **efficient knowledge updates** for LLMs, enabling broader applications and future advancements in the field.

## 2 PRELIMINARY

### 2.1 AUTOREGRESSIVE LANGUAGE MODEL

An autoregressive large language model (LLM) predicts the next token $\boldsymbol{x}$ in a sequence based on the preceding tokens. Specifically, the hidden state of $\boldsymbol{x}$ at layer $l$ within the model, denoted as $\boldsymbol{h}^l$, can be calculated as:

$$\boldsymbol{h}^l = \boldsymbol{h}^{l-1} + \boldsymbol{a}^l + \boldsymbol{m}^l, \quad \boldsymbol{m}^l = \boldsymbol{W}^l_{\text{out}}\, \sigma(\boldsymbol{W}^l_{\text{in}}\, \gamma(\boldsymbol{h}^{l-1} + \boldsymbol{a}^l)\,), \tag{1}$$

where $\boldsymbol{a}^l$ and $\boldsymbol{m}^l$ represent the outputs of the attention block and the feed-forward network (FFN) layer, respectively; $\boldsymbol{W}^l_{\text{in}}$ and $\boldsymbol{W}^l_{\text{out}}$ are the weight matrices of the FFN layers; $\sigma$ is the non-linear activation function, and $\gamma$ denotes the layer normalization. Following Meng et al. (2022), we express the attention and FFN modules in parallel here.

It is worth noting that $\boldsymbol{W}^l_{\text{out}}$ within FFN layers is often interpreted as a linear associative memory, functioning as key-value storage for information retrieval (Geva et al., 2021). Specifically, if the knowledge stored in LLMs is formalized as $(s, r, o)$ — representing subject $s$, relation $r$, and object $o$ (*e.g.*, $s =$ "The latest Olympic Game", $r =$ "was held in", $o =$ "Paris") — $\boldsymbol{W}^l_{\text{out}}$ associates a set of input keys $\boldsymbol{k}$ encoding $(s, r)$ with corresponding values $\boldsymbol{v}$ encoding $(o)$. That is,

$$\underbrace{\boldsymbol{m}^l}_{\boldsymbol{v}} = \boldsymbol{W}^l_{\text{out}}\, \underbrace{\sigma(\boldsymbol{W}^l_{\text{in}}\, \gamma(\boldsymbol{h}^{l-1} + \boldsymbol{a}^l)\,)}_{\boldsymbol{k}}. \tag{2}$$

This interpretation has inspired most model editing methods to modify the FFN layers for knowledge updates (Hase et al., 2023; Li et al., 2024a; Hu et al., 2024). For simplicity, we use $\boldsymbol{W}$ to refer to $\boldsymbol{W}^l_{\text{out}}$ in the following sections.

### 2.2 MODEL EDITING IN LLMS

Model editing aims to update knowledge stored in LLMs through a single edit or multiple edits (*i.e.*, sequential editing). Each edit modifies the model parameters $\boldsymbol{W}$ by adding a perturbation $\boldsymbol{\Delta}$ in locate-then-edit paradigm. Specifically, suppose each edit needs to update $u$ pieces of knowledge in the form of $(s, r, o)$. The perturbed $\boldsymbol{W}$ is expected to associate $u$ new $\boldsymbol{k}$-$\boldsymbol{v}$ pairs, where $\boldsymbol{k}$ and $\boldsymbol{v}$ encode $(s, r)$ and $(o)$ of the new knowledge, respectively. Let $\boldsymbol{W} \in \mathbb{R}^{d_1 \times d_0}$, where $d_0$ and $d_1$ represent the dimensions of the FFN's intermediate and output layers. Then, we can stack these keys and values into matrices following:

$$\boldsymbol{K}_1 = [\boldsymbol{k}_1\, |\boldsymbol{k}_2| \ldots |\, \boldsymbol{k}_u] \in \mathbb{R}^{d_0 \times u}, \quad \boldsymbol{V}_1 = [\boldsymbol{v}_1\, |\boldsymbol{v}_2| \ldots |\, \boldsymbol{v}_u] \in \mathbb{R}^{d_1 \times u}, \tag{3}$$

where the subscripts of $\boldsymbol{k}$ and $\boldsymbol{v}$ represent the index of the to-be-updated knowledge. Based on these, the objective can be expressed as:

$$\boldsymbol{\Delta} = \arg\min_{\tilde{\boldsymbol{\Delta}}} \left\| (\boldsymbol{W} + \tilde{\boldsymbol{\Delta}})\boldsymbol{K}_1 - \boldsymbol{V}_1 \right\|^2, \tag{4}$$

where $\|\cdot\|^2$ denotes the sum of the squared elements in the matrix.

Additionally, let $\boldsymbol{K}_0$ and $\boldsymbol{V}_0$ represent the matrices formed by stacking the $\boldsymbol{k}$ and $\boldsymbol{v}$ of the preserved knowledge. Current methods (Meng et al., 2023; Gu et al., 2024) typically incorporate the error involving $\boldsymbol{K}_0$ and $\boldsymbol{V}_0$ to preserve it, as follows:

$$\boldsymbol{\Delta} = \arg\min_{\tilde{\boldsymbol{\Delta}}} \left( \left\| (\boldsymbol{W} + \tilde{\boldsymbol{\Delta}})\boldsymbol{K}_1 - \boldsymbol{V}_1 \right\|^2 + \left\| (\boldsymbol{W} + \tilde{\boldsymbol{\Delta}})\boldsymbol{K}_0 - \boldsymbol{V}_0 \right\|^2 \right). \tag{5}$$

Since $\boldsymbol{K}_0$ and $\boldsymbol{V}_0$ encode the preserved knowledge in LLMs, we have $\boldsymbol{W}\boldsymbol{K}_0 = \boldsymbol{V}_0$ (*cf.* Eqn. 2). Thus, by applying the normal equation (Lang, 2012), if the closed-form solution of Eqn. 5 exists, it can be written as:

$$\boldsymbol{\Delta} = (\boldsymbol{V}_1 - \boldsymbol{W}\boldsymbol{K}_1)\, \boldsymbol{K}_1^T \left( \boldsymbol{K}_0 \boldsymbol{K}_0^T + \boldsymbol{K}_1 \boldsymbol{K}_1^T \right)^{-1}. \tag{6}$$

Although $\boldsymbol{K}_0$ is difficult to obtain directly since we hardly have access to the LLM's full extent of knowledge, it can be estimated using abundant text input (Meng et al., 2023). In practical applications, $100,000$ $(s, r, o)$ triplets from Wikipedia are typically randomly selected to encode $\boldsymbol{K}_0$ (Meng et al., 2023), making $\boldsymbol{K}_0$ a high-dimensional matrix with $100,000$ columns (*i.e.*, $\boldsymbol{K}_0 \in \mathbb{R}^{d_0 \times 100,000}$). See Appendix B.1 for detailed implementation steps.